GUÍA TÉCNICA DE DESARROLLO

Monolito Spring Boot — CasaJoJoApp

Arquitectura, Servicios, Flujo entre Capas y Guía de Implementación Backend

Propósito del Documento

Este manual tiene como objetivo guiar al equipo de desarrollo en la construcción del backend de **CasaJoJoApp**. Explica el funcionamiento de la arquitectura Monolito Modular en Java con Spring Boot, la responsabilidad de sus servicios principales, la integración del menú con Google Sheets y el flujo estricto de datos entre capas.

1. Concepto de Monolito Modular

El sistema se despliega como un **Monolito Modular**: un único archivo ejecutable (JAR) en Java con Spring Boot que centraliza toda la lógica de negocio en un solo proceso runtime, pero manteniendo una separación estricta de paquetes lógicos por dominio (Usuarios, Menú/Inventario, Pedidos/Cocina y Modificaciones).



2. Ventajas para el Proyecto

- **Despliegue Único:** Un solo contenedor o servidor ejecuta toda la aplicación sin sobrecostos de red.
- **Transacciones Simples:** Garantía ACID directa mediante la anotación `@Transactional` de Spring.
- **Mantenibilidad:** Organización modular que facilita refactorizar o extraer servicios en el futuro si la demanda crece.

3. Estructura de Paquetes del Código Fuente

Todo el desarrollo debe estructurarse rigurosamente bajo el paquete base `com.casajojoapp.backend`:

```
com.casajojoapp.backend
├── config/            --> SecurityConfig, OpenApiConfig, CorsConfig
├── enums/             --> EstadoPedido, CategoriaItem, RolUsuario, TipoIdentificacion
├── entity/            --> Usuario, Pedido, DetallePedido, ItemMenu, MenuDiario
├── dto/
│   ├── request/       --> CrearPedidoRequest, LoginRequest, SolicitarModificacionRequest
│   └── response/      --> PedidoResponse, MenuDiarioResponse, ResumenCocinaResponse
├── repository/        --> UsuarioRepository, PedidoRepository, ItemMenuRepository
├── service/           --> UsuarioService, MenuService, PedidoService, ModificacionPedidoService
│   └── impl/          --> UsuarioServiceImpl, MenuServiceImpl, PedidoServiceImpl
├── controller/        --> AuthController, MenuController, PedidoController
└── exception/         --> GlobalExceptionHandler, BusinessRuleException
```

4. Flujo Detallado de una Petición HTTP

Capa	Componente	Responsabilidad Específica
1. Presentación	`Controller`	Recibe el JSON, valida la estructura con `@Valid` sobre DTOs y mapea códigos de respuesta HTTP. No contiene reglas de negocio.
2. Negocio	`Service` / `ServiceImpl`	Aplica validaciones de negocio, controla el aislamiento transaccional (`@Transactional`) y transforma Entities a DTOs.
3. Persistencia	`Repository`	Ejecuta consultas a MySQL mediante Spring Data JPA. Aplica bloqueos pesimistas para proteger el stock.
4. Modelo	`Entity`	Mapeo relacional objeto-base de datos (OR/M Hibernate) protegido internamente sin exponerse a la API.

> **Regla de Oro de Separación**
>
> Las clases `Entity` de JPA **nunca** deben retornarse directamente en los Controllers. Siempre deben convertirse a objetos `DTO Response` para evitar fuga de contraseñas o bucles infinitos por referencias circulares.

5. Módulos de Servicios Principales (Parte 1)

Módulo 1: UsuarioService (Autenticación e Identidad Mixta)

Ubicación: `com.casajojoapp.backend.service.impl.UsuarioServiceImpl`

Administra el registro, autenticación y perfiles de acceso. Soporta tres modalidades de cliente:

- **ANONIMA:** Pedidos rápidos ingresando solo nombre y teléfono al confirmar.
- **TEMPORAL:** Perfil con identificador UUID generado desde la App Android para guardar historial local.
- **FORMAL:** Registro con email y contraseña cifrada mediante `BCryptPasswordEncoder`.

Métodos clave: `login(LoginRequest)`, `registrarCliente(RegistroRequest)`, `obtenerPerfil(Long id)`.

Módulo 2: MenuService (Integración con Google Sheets & Inventario Local)

Ubicación: `com.casajojoapp.backend.service.impl.MenuServiceImpl`

Aunque la definición inicial de productos y menú proviene de Google Sheets, el backend debe mantener una **copia local sincronizada en MySQL** para gestionar el stock en tiempo real.

- **Sincronización Sheets ➔ MySQL:** Un servicio lee la API de Google Sheets y actualiza la tabla `item_menu` asignando el stock inicial del día.
- **Control de Stock Operativo:** El descuento de inventario durante las compras se efectúa directamente sobre MySQL con alta velocidad y soporte concurrente.

Métodos clave: `sincronizarConGoogleSheets()`, `obtenerMenuVigente()`, `descontarStock(itemId, cantidad)`.

Beneficio de la Caché Local en MySQL

No se consulta directamente Google Sheets al crear un pedido. Esto evita latencias externas altas y fallos si la API de Google alcanza sus límites de cuota.

6. Módulos de Servicios Principales (Parte 2)

Módulo 3: PedidoService (Lógica de Pedidos y Cola FIFO de Cocina)

Ubicación: `com.casajojoapp.backend.service.impl.PedidoServiceImpl`
Representa el núcleo operativo de CasaJoJoApp. Maneja la creación, cobro y ciclo de vida de los pedidos.

- **Creación Transaccional:** Verifica que el menú esté `ABIERTO`, bloquea y descuenta el stock, calcula precios en el servidor y congela el `precio_historico` en el detalle.
- **Cola FIFO para Cocina:** Consulta pedidos activos (`RECIBIDO`, `EN_PREPARACION`) ordenados prioritariamente por `hora_retiro ASC`.
- **Máquina de Estados:** Habilita la transición controlada: `RECIBIDO` ➔ `EN_PREPARACION` ➔ `LISTO_PARA_RETIRO` ➔ `ENTREGADO`.

Métodos clave: `crearPedido(CrearPedidoRequest)`, `obtenerColaCocina()`, `actualizarEstado(id, nuevoEstado)`.

Módulo 4: ModificacionPedidoService (Ajustes Post-Envío)

Ubicación: `com.casajojoapp.backend.service.impl.ModificacionPedidoServiceImpl`
Permite gestionar requerimientos del cliente para modificar pedidos antes de su preparación.

- **Solicitud:** El cliente registra la petición siempre que el pedido esté en estado `RECIBIDO`.
- **Resolución Administrativa:** El personal aprueba o rechaza la solicitud. Si se aprueba, se reajusta automáticamente el stock y se recalcula el total.

Métodos clave: `solicitarModificacion(request)`, `aprobarORechazarModificacion(id, aprobada)`.

Inmutabilidad Financiera

Cualquier modificación posterior en los precios del menú o en Sheets **jamás altera** los importes congelados en `DetallePedido.precio_historico` de comandas ya emitidas.

7. Contrato DTO vs. Entidades JPA

Para ilustrar la separación entre la capa de API externa y la base de datos interna, se especifica la estructura de los objetos de transferencia de datos (DTOs):

Estructura: CrearPedidoRequest (DTO de Entrada)

```
{
  "usuarioId": 105,               // Opcional (null si es anónimo)
  "nombreCliente": "Juan Pérez",  // Obligatorio para anónimos
  "telefonoContacto": "+56912345678",
  "horaRetiroSolicitada": "13:30", // Formato HH:mm (mínimo 20 min en el futuro)
  "items": [
    { "itemMenuId": 12, "cantidad": 2 },
    { "itemMenuId": 45, "cantidad": 1 }
  ]
}
```

Estructura: PedidoResponse (DTO de Salida)

```
{
  "id": 482,
  "codigoSeguimiento": "CJ-98F2",
  "estado": "RECIBIDO",
  "horaRetiro": "13:30",
  "montoTotal": 8500.00,        // Calculado por el servidor
  "fechaCreacion": "2026-10-19T10:15:00Z",
  "detalles": [
    { "nombreItem": "Menú Ejecutivo Proteína", "cantidad": 2, "precioUnitario": 3500.00 },
    { "nombreItem": "Bebida 350ml", "cantidad": 1, "precioUnitario": 1500.00 }
  ]
}
```

8. Control Global de Excepciones

GlobalExceptionHandler (@ControllerAdvice)

Captura excepciones del sistema (`BusinessRuleException`, `ResourceNotFoundException`) y retorna respuestas JSON estandarizadas con códigos HTTP semánticos (400, 404, 409, 422), **sin exponer trazas internas de la base de datos**.